TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A PROPOSAL

ON

MOTIONBRIDGE: A MULTI-CAMERA NEURAL NETWORK FRAMEWORK FOR REAL-TIME HUMAN ARM MOTION REPLICATION AND DATASET GENERATION

Submitted By:

Pratik Khatiwada (THA079BEI025)

Preeti Rijal (THA079BEI028)

Sampada Ghimire (THA079BEI036)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

June , 2026

ACKNOWLEDGEMENT

We sincerely wish to thank our project supervisor, **Dr. Binod Sapkota**, for his guidance, helpful advice, and continued support in the project.

We sincerely thank **Asst. Prof. Umesh Kanta Ghimire**, Head of the Department, and the whole Department of Electronics and Computer Engineering for providing us with the required facilities and a conducive learning atmosphere.

We thank all our faculty members, friends, and all the other individuals, both directly or indirectly involved in this project for giving us their valuable suggestions and supportive encouragement.

Pratik Khatiwada (THA079BEI025)
Preeti Rijal (THA079BEI028)
Sampada Ghimire (THA079BEI036)

ABSTRACT

MotionBridge is a low-cost, real-time, multi-camera framework designed for human arm motion capture, simulation replication, and robotic dataset generation without relying on expensive depth sensors or wearable hardware. Utilizing three standard, calibrated RGB webcams, the system extracts 2D joint coordinates and 21-point hand landmarks per frame using MediaPipe Pose and MediaPipe Hands. These 2D keypoints are fused via a confidence-weighted triangulation algorithm into accurate 3D metric world coordinates. The reconstructed positions are mapped in real time to a simulated robotic arm inside a PyBullet physics environment using Inverse Kinematics (IK). Operating efficiently on standard CPU hardware, the pipeline simultaneously renders a live visual replication and exports a structured CSV dataset containing 3D joint trajectories, computed angles, and timestamps to support downstream robotic imitation learning.

Keywords: *Motion Capture, Multi-Camera Triangulation, MediaPipe, PyBullet Simulation, Inverse Kinematics, Robotic Dataset Generation, Computer Vision, Real-Time Systems.*

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

2D	Two-Dimensional
3D	Three-Dimensional
CPU	Central Processing Unit
CSV	Comma-Separated Values
DLS	Damped Least Squares
DRL	Deep Reinforcement Learning
FPS	Frames Per Second
GPU	Graphics Processing Unit
HPE	Human Pose Estimation
HRC	Human-Robot Collaboration
IK	Inverse Kinematics
IMU	Inertial Measurement Unit
NPR	Nepalese Rupee
PC	Personal Computer
RAM	Random Access Memory
RGB	Red Green Blue
RGB-D	Red Green Blue - Depth
SVD	Singular Value Decomposition
URDF	Unified Robot Description Format
USB	Universal Serial Bus
VAR	Video Assistant Referee
VLM	Vision-Language Model

1. INTRODUCTION

1.1. Background

The ability to capture, interpret, and replicate human motion has long been a fundamental challenge in robotics, computer vision, and human-computer interaction. Traditional motion capture systems rely on expensive marker-based setups, wearable Inertial Measurement Unit (IMU), or specialized depth sensors such as the Microsoft Kinect and Intel RealSense, all of which impose significant hardware constraints and deployment costs. Recent advances in deep learning-based pose estimation, particularly lightweight frameworks such as MediaPipe developed by Google, have enabled accurate real-time extraction of human joint coordinates from standard RGB video without specialized hardware.

Simultaneously, physics-based simulation platforms such as PyBullet have matured into robust environments for replicating articulated body motion, providing physically accurate rendering of joint movement and a foundation for generating structured motion datasets. The convergence of these two technologies vision-based pose estimation and physics-based simulation presents an opportunity to build affordable, accessible motion capture systems that require only standard cameras.

In the context of industrial robotics and assistive automation, large-scale datasets of human arm movement are essential for training robotic manipulators to perform tasks mimicking natural human motion. However, the cost and complexity of existing data collection pipelines remain a barrier to widespread dataset generation. A vision-based, multi-camera system capable of capturing full arm-to-fingertip motion in real-time and replicating it in simulation addresses this barrier directly.

1.2. Motivation

Robotic arms capable of replicating human arm motion have significant potential in industrial automation, assistive robotics, teleoperation, and rehabilitation engineering. A core requirement for training such systems is access to large, diverse datasets of human arm trajectories annotated with joint coordinates and angles. Existing approaches to generating such datasets either rely on expensive hardware (RGB-D sensors, IMU suits) or produce offline, non-real-time outputs that are difficult to scale.

The motivation for MotionBridge arises from the absence of a low-cost, real-time, vision-only pipeline that combines multi-camera capture, neural network-based pose estimation, three-dimensional reconstruction, and physics simulation into a single integrated system. By using only standard USB webcams and open-source software, the proposed system can be deployed in resource-constrained environments such as

academic laboratories, small research groups, and developing-world institutions where expensive motion capture infrastructure is unavailable.

Furthermore, no existing reviewed work captures both full arm joint trajectories and detailed finger joint coordinates (21 points per hand) from standard RGB cameras and simultaneously replicates arm movement in simulation. MotionBridge addresses this gap, producing a richer dataset than currently available vision-based approaches.

1.3. Problem Statement

Existing human arm motion capture systems for robotic dataset generation suffer from one or more of the following limitations: reliance on expensive depth-sensing hardware, dependence on wearable sensors, absence of real-time simulation replication, operation on monocular camera inputs that cannot resolve depth accurately, or omission of finger-level joint data. These constraints prevent scalable and affordable motion dataset generation for robotic arm training.

This project addresses the problem of building a low-cost, real-time system that captures three-dimensional human arm and finger joint motion using only standard RGB cameras, replicates the captured motion in a physics-based simulation environment, and produces a structured dataset of joint coordinates and angles for downstream robotic arm training without requiring specialized depth sensors, wearable hardware, or GPU-accelerated inference.

1.4. Objectives

- To implement a vision-only system that extracts real-time 2D arm and hand landmarks from three standard RGB webcams and fuses them into 3D world coordinates via confidence-weighted triangulation.
- To map reconstructed 3D joint trajectories onto a simulated PyBullet robotic arm using Inverse Kinematics while simultaneously exporting a structured CSV dataset for robotic training.

1.5. Project Scope

1.5.1. Capabilities

- Real-time capture of human arm movement (shoulder, elbow, wrist) using three standard USB webcams.
- Detection of 21 finger joint landmarks per hand using MediaPipe Hands in addition to full arm keypoints.

- Three-dimensional joint coordinate reconstruction via confidence-weighted triangulation using calibrated multi-camera geometry.
- Real-time replication of arm movement in PyBullet physics simulation using Inverse Kinematics.
- Generation of a structured dataset containing per-frame 3D joint coordinates, joint angles, and timestamps.
- Occlusion robustness through multi-camera redundancy when one camera's view is blocked, remaining cameras compensate.

1.5.2. Limitations

- Finger joint movement is captured and recorded in the dataset but is not replicated in PyBullet simulation, as standard arm URDF models do not include articulated finger structures.
- The system is limited to single-person capture; multi-person simultaneous tracking is outside scope.
- Accuracy depends on initial camera calibration quality; dynamic camera repositioning requires recalibration.
- The system does not include robot arm training or policy learning; it generates the dataset only.
- Performance may degrade under extreme lighting conditions or significant self-occlusion not resolvable by three cameras.
- The system does not support full-body motion capture only the arm from shoulder to fingertip.

1.6. Problem Applcations

The dataset and simulation output produced by MotionBridge have direct applications in the following domains:

- **Industrial Automation:** Providing training data for robotic arms performing labour-intensive factory tasks such as assembly, sorting, and load-carrying that mimic human arm motion.
- **Assistive Robotics:** Generating motion trajectories for robotic prosthetics and assistive devices that replicate natural human arm and hand movement.
- **Teleoperation:** Serving as a reference system for mapping human operator arm motion to remote robotic manipulators.
- **Rehabilitation Engineering:** Capturing and analysing patient arm movement for physiotherapy assessment and progress monitoring.

- **Imitation Learning Research:** Supplying labelled demonstration datasets for behavioral cloning and inverse reinforcement learning pipelines.
- **Simulation-Based Training:** Using the PyBullet output as a reference motion environment for reinforcement learning agents.

2. LITERATURE REVIEW

2.1. Related Works

Human Pose Estimation (HPE) forms the foundational stage of any vision-based motion capture system. Kim et al. [1] demonstrated the effectiveness of MediaPipe Pose in a two-stage pipeline running on a single-board computer mounted on a mobile robot, achieving a mean joint coordinate difference of 0.097 m and an average angle difference of 10.017 degrees per joint across six daily poses. The study confirmed that MediaPipe provides a practical balance between computational cost and accuracy for real-time embedded applications, directly supporting its adoption as the pose estimation backbone in the proposed system.

Sarkar et al. [2] implemented a multi-camera system for close-proximity human-robot collaboration in construction environments, using MediaPipe Pose to extract 15 keypoints per frame from each camera view and converting them to 3D coordinates through rotation, translation, and homogeneous transformation matrices derived from known camera positions and orientations. This work is the closest architectural match to the proposed pipeline, validating the feasibility of combining MediaPipe with a multi-camera setup for real-world 3D joint reconstruction without specialized depth hardware.

Zell et al. [3] presented a multimodal human motion dataset containing 51,051 poses from 71 persons, obtained by extracting 2D keypoints using MediaPipe Human Pose Landmarker from virtual multi-camera images and computing corresponding 3D keypoints by linear triangulation. Their dataset was explicitly intended for transforming computer vision pose solutions into biomechanical simulations, constituting a direct proof of concept for the MediaPipe-plus-triangulation pipeline proposed in MotionBridge.

Bultmann and Behnke [4] proposed a distributed multi-camera architecture where 2D joint detection is performed locally at each smart edge sensor and only the semantic skeleton representation is transmitted to a central backend for 3D triangulation. Their system achieves real-time 3D pose estimation while significantly reducing network bandwidth, and incorporates prior human skeleton knowledge into the triangulation backend to improve robustness an architectural approach directly relevant to the distributed and real-time requirements of the proposed system.

Bragagnolo et al. [5] addressed the known limitation of standard triangulation under occlusion by performing multi-view fusion on 3D skeletons from monocular methods rather than fusing noisy 2D features, applying reprojection error optimization with limb-length symmetry constraints. Their method significantly outperformed standard

triangulation in human-robot collaboration scenarios with heavy occlusion, identifying a clear accuracy upgrade path for the proposed system beyond its baseline triangulation implementation.

Leroy et al. [6] extended multi-view 3D pose estimation to uncalibrated camera setups through Probabilistic Triangulation, using Monte Carlo sampling to iteratively estimate camera pose distributions. Their experiments on Human3.6M and CMU Panoptic benchmarks demonstrated that accurate 3D reconstruction is achievable without fixed camera calibration, which is relevant when the proposed system is deployed in environments where precise camera recalibration is impractical.

Liang et al. [7] proposed the most directly related simulation-based work, integrating 3D HPE with PyBullet for robotic arm motion imitation. Their pipeline extracts human arm motion from video using a Strided Transformer network, retargets the human morphology to a robot URDF model, generates reference trajectories in PyBullet using its built-in Inverse Kinematics solver, and applies deep reinforcement learning for policy training. However, their system operates offline on monocular pre-recorded video and targets robot policy training rather than dataset generation, distinguishing it from the real-time multi-camera focus of MotionBridge.

Jiang et al. [8] demonstrated real-time multi-person arm motion imitation using a single RGB-D camera and an improved retargeting algorithm, achieving consistent arm configuration and precise end-effector tracking for multi-robot collaboration tasks. While their system confirms that real-time arm motion replication is feasible, it relies on expensive RGB-D hardware for depth acquisition, whereas MotionBridge targets equivalent real-time performance using only standard RGB cameras with computational triangulation, substantially lowering the deployment cost.

Faria et al. [9] presented a teleoperation framework combining MediaPipe landmark detection with an Intel RealSense RGB-D camera for 3D joint reconstruction, mapping the results to a robot arm's kinematic constraints in real-time. Their work explicitly noted that vision-only estimation remains limited for axial rotations such as elbow and wrist yaw under occlusions a limitation that the multi-camera triangulation approach in MotionBridge is specifically designed to mitigate by providing redundant viewpoints.

Moya-Alcover et al. [10] presented seven motion capture datasets of professional industrial gestures performed by skilled craftsmen and factory operators, recorded using wearable inertial sensors. Their work is motivated by the same industrial and labour application domain described in MotionBridge capturing arm motion for tasks involving factory work and load-carrying but relies on instrumented suits rather than

vision-based capture, introducing hardware constraints that the proposed camera-based system avoids entirely.

2.2. Existing Systems and Limitations

A comparative analysis of the reviewed works reveals consistent gaps across three dimensions: hardware cost, depth resolution method, and system completeness. Table 2-1 summarises the key attributes of existing systems against the proposed MotionBridge system.

Table 2-1. Comparison of Related Works with Proposed System

Work	**Year**	**Camera Setup**	**Depth Method**	**Simulation**	**Primary Goal**
Liang et al.	2024	Monocular	Transformer	PyBullet	DRL Training
Jiang et al.	2025	Single RGB-D	Hardware	Real Robot	Robot Control
Faria et al.	2025	Single RGB-D	Hardware	Real Robot	Teleoperation
Sarkar et al.	2022	Multi RGB	Homogeneous	None	HRC Safety
Bragagnolo et al.	2024	Multi RGB	Multi-view Fusion	None	Pose Accuracy
Bultmann & Behnke	2021	Multi (edge)	Triangulation	None	Real-time 3D
Moya-Alcover et al.	2023	IMU-based	Inertial	None	Dataset
Zell et al.	2024	Virtual Multi	Triangulation	None	Dataset
MotionBridge	**2026**	**Multi RGB**	**Conf.-Weighted Tri.**	**PyBullet**	**Simulation + Dataset**

Systems that achieve real-time arm replication (Jiang et al., Faria et al.) depend on hardware depth sensors costing $300 or more. Systems using standard cameras and triangulation (Sarkar et al., Zell et al., Bultmann and Behnke) omit the simulation replication stage entirely. The only system using PyBullet for arm motion replication (Liang et al.) operates offline on monocular input and produces no structured dataset. No reviewed work captures finger-level joint data alongside arm joints from standard RGB cameras. MotionBridge uniquely combines all four capabilities multi-camera RGB capture, real-time pose estimation, confidence-weighted triangulation, PyBullet simulation replication, and full arm-to-fingertip dataset generation in a single low-cost pipeline.

3. REQUIREMENTS

3.1. Hardware Requirements

- **USB Webcams × 3:** Standard 1080p USB webcams (e.g., Logitech C270 or equivalent), positioned at front, left, and right angles around the capture area. Minimum 30 fps required for real-time operation.
- **Camera Mounts / Tripods × 3:** Fixed mounting hardware to ensure stable, calibrated camera positions during capture sessions.
- **Checkerboard Calibration Target:** Printed A4 checkerboard pattern (9×6 inner corners) for OpenCV camera calibration.
- **Laptop / Desktop PC:** Minimum Intel Core i5 (8th generation or above), 8 GB RAM. No GPU required MediaPipe and PyBullet both run on CPU for this pipeline.
- **USB Hub (optional):** If the host machine has insufficient USB ports for three simultaneous camera connections.

3.2. Software Requirements

- **Programming Language:** Python 3.9 or above.
- **Pose Estimation:** MediaPipe 0.10+ (`mediapipe` Python package) - for real-time arm and hand landmark detection.
- **Computer Vision:** OpenCV 4.8+ (`opencv-python`) - for camera capture, calibration, and triangulation.
- **Physics Simulation:** PyBullet (`pybullet`) - for IK computation and real-time arm simulation rendering.
- **Numerical Computing:** NumPy (`numpy`) - for matrix operations in triangulation and IK.
- **Data Storage:** Python `csv` module or Pandas (`pandas`) - for structured dataset writing.
- **Development Environment:** Visual Studio Code on Windows 10/11 or Ubuntu 22.04.
- **Version Control:** Git / GitHub - for source code management.

4. PROPOSED SYSTEM/ METHODOLOGY

4.1. System Overview

The MotionBridge framework is designed as a modular, hardware-agnostic software pipeline that translates real-time multi-view human upper limb movements directly into physical simulation joint trajectories, automating behavioral cloning dataset factory operations. The structural design isolates computer vision feature extraction in Phase I from physical joint simulation and serialization loops in Phase II. This analytical execution pipeline bypasses heavy iterative matrix math during runtime to achieve real-time performance, continuously serializing the resulting mathematically predicted joint angles and tracking coordinates. The distribution of components, interfaces, and boundary layers across the primary execution phases of the framework is illustrated in Figure 4-1.

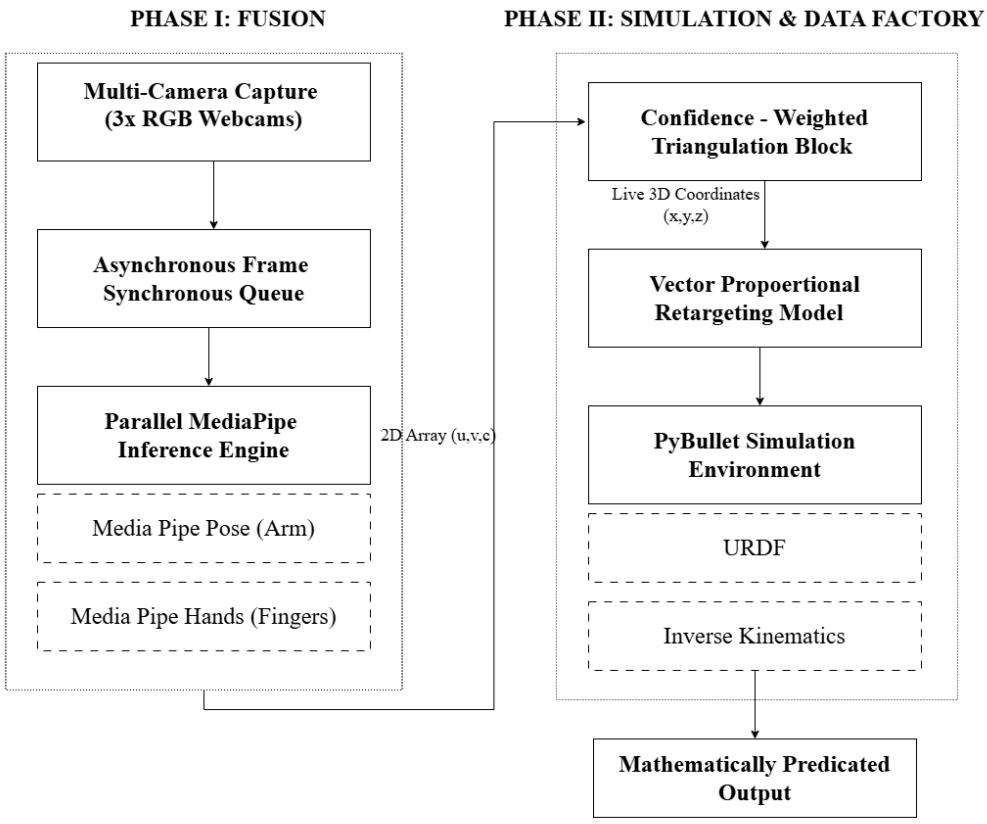

Figure 4-1. Block diagram of the MotionBridge framework.

4.1.1. Phase I: Multi-View Spatial Data Acquisition & Fusion

Geometric Camera Calibration

To eliminate lens distortion artifacts and define structural world space mapping variables, three consumer-grade RGB 1080p webcams are fixed on rigid tripods encircling the operator workspace. Geometric evaluation relies on OpenCV's checkerboard target matrix configuration (9×6 internal corners). The algorithm computes the individual camera intrinsic parameter matrix K_i and the camera extrinsic spatial orientation matrix $[R_i|t_i]$, defining a permanent structural projection matrix P_i per camera view i:

$$P_i = K_i \cdot [R_i|t_i] \quad \text{where } i \in \{1, 2, 3\} \tag{4-1}$$

Multi-Threaded Frame Synchronization

Asynchronous USB bus clock jitters create temporal variations across video streams, compromising spatial triangulation. To mitigate this issue, individual camera captures are isolated onto separate thread loops managed via a central thread queue. Incoming frames are assigned system arrival timestamps (t_{cam}); a matching mechanism drops unaligned frame pairs, enforcing temporal consistency across views if and only if:

$$\Delta t = |t_{\text{cam},i} - t_{\text{cam},j}| < 15\,\text{ms} \quad \forall\ i, j \in \{1, 2, 3\} \tag{4-2}$$

Parallel Multi-Model Keypoint Extraction

Synchronized frames are concurrently transmitted to localized, independent instances of Google MediaPipe libraries. The frame processing loop targets parallel data extraction tasks:

1. **MediaPipe Pose Landmarker:** Extracts structural coordinates defining large-scale body vectors, focusing on the shoulder, elbow, and wrist nodes.
2. **MediaPipe Hands Mesh Model:** Tracks dense regional features by mapping 21 localized tracking landmarks per hand to monitor intricate gesturing profiles.

This combined parsing generates a continuous stream of two-dimensional pixel arrays $x_i = (u_i, v_i)$ paired with an associated, dynamic visibility validation metric $c_i \in [0, 1]$.

Confidence-Weighted Triangulation Calculus

Standard linear triangulation using Singular Value Decomposition ($AX = 0$) breaks down under severe occlusion conditions common to single-user movement. MotionBridge

counteracts viewpoint dropouts by adjusting the projection matrix influence dynamically using MediaPipe's live visibility score (c_i). This calculation weights down occluded views, computing accurate 3D metric world coordinates (X_{3D}):

$$X_{3D} = \frac{\sum_{i=1}^{3} c_i \cdot f(P_i, x_i)}{\sum_{i=1}^{3} c_i} \tag{4-3}$$

4.1.2. Phase II: Simulation Mapping, Kinematic Processing, & Automated Dataset Generation

Vector Proportional Retargeting

Because human bone proportions vary significantly from robot dimensions, raw metric world values (X_{3D}) cannot be directly mapped to joint targets. To address this, human tracking segments are evaluated as directional spatial vectors:

$$\vec{V}_{\text{segment}} = X_{3D,\text{ child}} - X_{3D,\text{ parent}} \tag{4-4}$$

These vectors are proportionally normalized and scaled according to the mechanical link lengths specified within the robot URDF asset file, preventing joint hyper-extension and structural calculation drift.

Articulated Kinematic Setup within PyBullet

The simulation environment is built inside the PyBullet physics engine utilizing the Kuka IIWA URDF manipulator model. This environment provides physically accurate rendering of joint movements and maps the mathematical spatial features to a standardized robotic configuration.

Damped Least Squares Inverse Kinematics

To map target positions to joint rotations without encountering mathematical divisions by zero near arm workspace boundaries (singularities), PyBullet's internal solver is configured to evaluate joint states via a Damped Least Squares (DLS) optimization algorithm:

$$\theta = \text{pybullet.calculateInverseKinematics}(\text{bodyId}, \text{jointId}, X_{\text{wrist}}) \tag{4-5}$$

The calculated joint configuration array θ updates the virtual robot model at a consistent execution rate via motor position control commands.

Automated Data Factory Serialization

While maintaining live display rendering, the system operates as an automated data logging factory. The processing loop writes synchronized parameters directly to a structured CSV format trajectory file. Each line records the spatial features and target joint angles for that frame:

$$\text{Record} = \{t, X_{\text{shoulder}}, X_{\text{elbow}}, X_{\text{wrist}}, X_{\text{fingers}}, \theta\} \tag{4-6}$$

The step-by-step runtime pipeline execution path is illustrated in Figure 4-2.

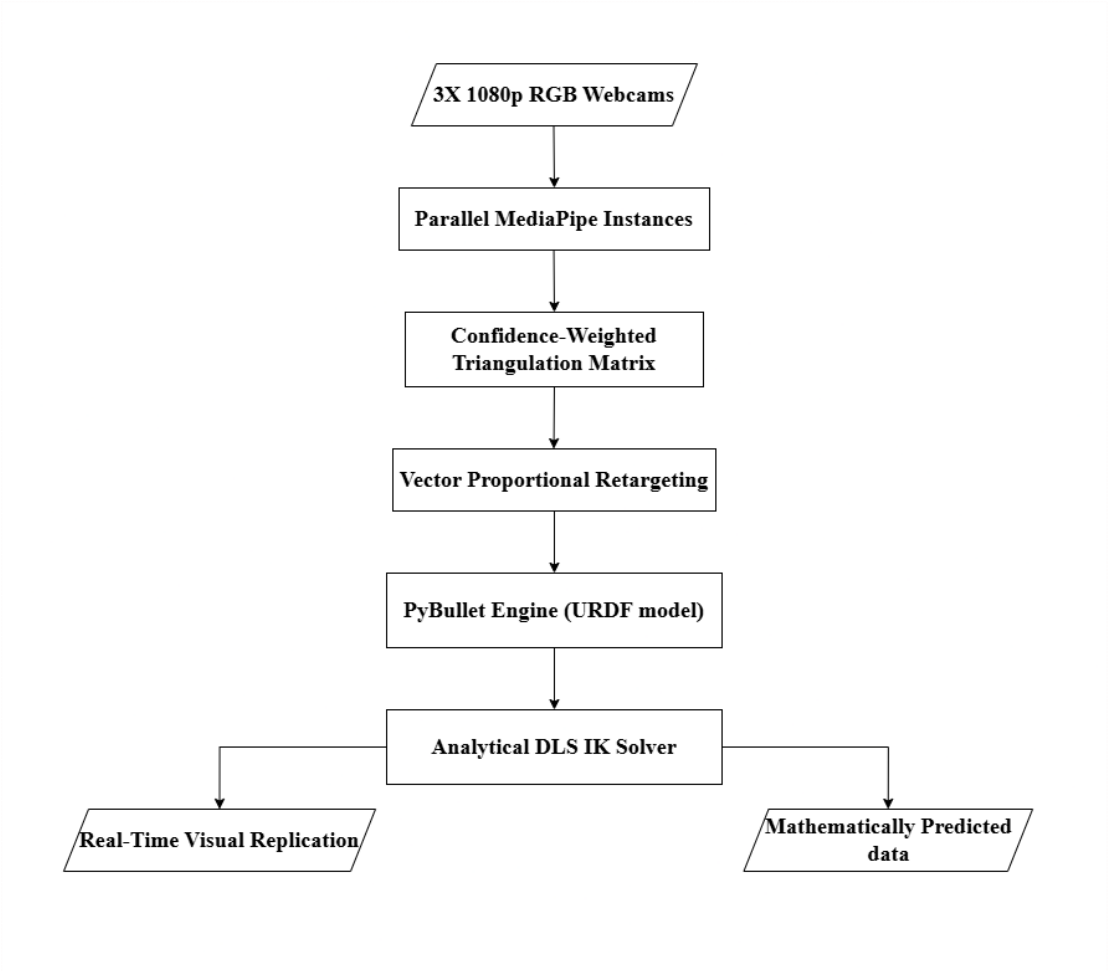

Figure 4-2. Procedural Flowchart outlining the runtime sequence.

4.2. System Robustness: Algorithmic Noise Mitigation

The Problem of Mathematical Trajectory Discontinuity

A prominent challenge within this pure computer vision triangulation setup is coordinate discontinuity across frames. Even with multi-view redundancy, sudden

body self-occlusion or spatial feature flickering causes quick spikes in calculated 3D targets (X_{3D}). Because standard analytical Inverse Kinematics (IK) evaluates frames independently, these small position jumps propagate directly into the output joint configurations (θ). This causes high-frequency mechanical jitter in simulation, which corrupts the dataset for smooth imitation learning.

To address these spatial jumps and build a curated, highly precise tracking resource, future iterations of this framework will investigate two distinct smoothing blocks implemented after the spatial triangulation step:

Proposed Extension A: Analytical Temporal Filtering (One Euro Filter)

To resolve tracking noise directly during runtime without adding significant CPU latency, we propose integrating an analytical adaptive low-pass filter, specifically a **One Euro Filter**. Unlike standard Butterworth filters that cause lag, or a traditional Kalman Filter that demands complex physical state tracking, the One Euro Filter dynamically adjusts its filtering cutoff frequency based on target velocity:

$$\alpha = \frac{1}{1 + \frac{\tau}{\Delta t}} \quad \text{where} \quad \tau = \frac{1}{2\pi \cdot (f_{\min} + \beta \cdot |\dot{X}_{3D}|)} \tag{4-7}$$

When target movement speed is low, the filter lowers the cutoff frequency to smooth out high-frequency sensor noise. When velocity ($|\dot{X}_{3D}|$) increases, it automatically scales up the cutoff to reduce lag, ensuring responsive tracking during quick movements.

Proposed Extension B: Deep Sequential Data Curation (LSTM Network)

For post-processing data curation or advanced prediction, we propose implementing a recurrent machine learning architecture based on **Long Short-Term Memory (LSTM)** neural networks.

Instead of treating frames as isolated events, an LSTM processes a moving sequence window of historical frames to learn continuous velocity patterns. This allows it to reconstruct coordinates when a joint is completely blocked from view. The proposed deep behavioral policy uses a sequential structure:

- **LSTM Layer Block:** Processes a sequence window of past frames to track movement direction and velocity trends.
- **MLP Projection Head:** Maps the processed temporal states to output clean, stable target joint angles (θ), removing noise from the dataset.

This network will be trained by minimizing a Mean Squared Error (MSE) loss function augmented with a temporal smoothness penalty:

$$\mathcal{L} = \frac{1}{N}\sum_{j=1}^{N}(\theta_{\text{predicted},j} - \theta_{\text{analytical},j})^2 + \gamma \sum_{j=2}^{N}(\theta_{\text{predicted},j} - \theta_{\text{predicted},j-1})^2 \qquad (4\text{-}8)$$

Here, γ is a regularization factor that penalizes sudden changes between consecutive steps, ensuring smooth, highly curated data sequences for training actual robotic systems.

5. EXPECTED RESULTS AND EVALUATION METRICS

5.1. Expected Results

The proposed MotionBridge system is expected to provide a low-cost and real-time framework for human arm motion capture, simulation replication, and robotic dataset generation using only standard RGB cameras and open-source software tools. By integrating MediaPipe-based landmark detection, multi-camera triangulation, and PyBullet simulation, the system is expected to generate accurate three-dimensional representations of human arm and hand motion without requiring specialized depth sensors or wearable devices.

The system is expected to successfully detect shoulder, elbow, wrist, and finger landmarks from multiple camera views and reconstruct their corresponding 3D world coordinates through confidence-weighted triangulation. The reconstructed arm motion is expected to be replicated in real time within a PyBullet simulation environment using inverse kinematics-based robotic arm control.

Simultaneously, the system is expected to generate a structured dataset containing timestamped 3D coordinates of arm and finger joints, along with corresponding robotic arm joint angles. This dataset is intended to support future applications in robotic imitation learning, teleoperation, rehabilitation engineering, and human–robot collaboration.

Furthermore, the multi-camera architecture is expected to improve robustness against partial occlusions by utilizing redundant viewpoints, thereby maintaining reliable landmark reconstruction even when one camera view is temporarily obstructed. The complete pipeline is expected to operate on standard CPU-based hardware while maintaining near real-time performance.

5.2. Evaluation Metrics

The performance of the proposed system will be evaluated using quantitative and qualitative metrics that assess landmark detection quality, three-dimensional reconstruction accuracy, real-time performance, simulation responsiveness, dataset quality, and robustness to occlusions.

5.2.1. Landmark Detection Performance

The effectiveness of MediaPipe Pose and MediaPipe Hands in detecting arm and hand landmarks will be evaluated using standard classification metrics based on correctly and incorrectly detected landmarks.

1. Precision

Precision measures the proportion of detected landmarks that are correctly identified.

$$Precision = \frac{TP}{TP + FP} \tag{5-1}$$

where:

- TP = True Positives
- FP = False Positives

2. Recall

Recall measures the proportion of actual landmarks that are successfully detected.

$$Recall = \frac{TP}{TP + FN} \tag{5-2}$$

where:

- FN = False Negatives

3. F1-Score

The F1-score provides a balanced measure of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{5-3}$$

Higher precision, recall, and F1-score values indicate better landmark detection performance.

5.2.2. Three-Dimensional Reconstruction Accuracy

The accuracy of the reconstructed 3D joint coordinates will be evaluated by comparing reconstructed positions with known reference measurements or predefined calibration poses.

The Mean Reconstruction Error will be calculated as:

$$E_{mean} = \frac{1}{N} \sum_{i=1}^{N} \left\| X_i - \hat{X}_i \right\| \tag{5-4}$$

where:

- X_i represents the reference joint position,
- $\hat{X}_i$ represents the reconstructed joint position,
- N represents the total number of evaluated joints.

Lower reconstruction error indicates better triangulation accuracy and more reliable 3D motion capture.

5.2.3. Real-Time Processing Performance

The computational efficiency of the proposed system will be evaluated using the following metrics:

- Frames Per Second (FPS)
- Average processing time per frame
- CPU utilization

Frames Per Second (FPS) will be computed as:

$$FPS = \frac{\text{Total Processed Frames}}{\text{Total Execution Time}} \tag{5-5}$$

The system is expected to maintain near real-time operation while processing multiple camera streams simultaneously.

5.2.4. Simulation Replication Latency

The responsiveness of the robotic arm simulation will be measured by calculating the delay between human motion capture and corresponding robotic arm movement within PyBullet.

Latency will be calculated as:

$$Latency = T_{simulation} - T_{capture} \tag{5-6}$$

where:

- $T_{capture}$ is the timestamp of landmark acquisition,
- $T_{simulation}$ is the timestamp of simulation update.

Lower latency values indicate better real-time motion replication performance.

5.2.5. Dataset Completeness

The quality and reliability of the generated dataset will be evaluated by measuring the percentage of frames containing valid joint information.

Dataset Completeness will be calculated as:

$$Completeness = \frac{\text{Valid Frames}}{\text{Total Frames}} \times 100 \tag{5-7}$$

A higher completeness percentage indicates successful landmark tracking and dataset generation throughout the recording session.

5.2.6. Occlusion Robustness

To evaluate the robustness of the proposed multi-camera framework, controlled occlusion experiments will be performed by partially blocking one or more camera views.

The following factors will be analyzed:

- Landmark detection continuity
- Reconstruction accuracy under occlusion
- Percentage of successfully reconstructed joints
- Simulation stability during occluded conditions

The multi-camera configuration is expected to reduce tracking failures and improve reconstruction reliability compared to monocular camera systems.

5.2.7. Overall System Evaluation

The overall effectiveness of MotionBridge will be assessed by combining the above metrics to determine whether the system can provide accurate, real-time human arm motion capture and robotic dataset generation using only low-cost RGB cameras and standard computing hardware.

Successful completion of the project will be demonstrated through:

- Reliable landmark detection,
- Accurate 3D joint reconstruction,
- Real-time robotic arm simulation,
- Low-latency operation,
- High dataset completeness,

- Robust performance under moderate occlusion conditions.

The results obtained from these evaluations will be used to assess the suitability of the proposed framework for future robotic imitation learning and human–robot interaction applications.

6. FEASIBILITY ANALYSIS

6.1. Technical Feasibility

The technical risk profile for the MotionBridge framework is exceptionally low due to the selection of mature, optimized, and well-documented open-source technologies. The system utilizes Google `MediaPipe` libraries, including Pose and Hands Landmarkers, for lightweight 2D feature extraction from video frames, `OpenCV` for spatial coordinate processing and triangulation, and `PyBullet` for kinematic simulation and motion mapping. These tools are widely adopted within the computer vision and robotics communities and have proven compatibility within standard Python environments.

The framework is designed to operate efficiently on commonly available hardware. Unlike traditional motion capture systems that require expensive depth sensors, specialized cameras, or dedicated GPU infrastructure, MotionBridge can function effectively on standard consumer-grade computers. The selected software components are optimized for CPU-based execution, making the system accessible to users with moderate hardware specifications such as an Intel Core i5 processor and 8 GB of RAM.

To ensure reliable motion reconstruction, the system incorporates confidence-based landmark processing techniques that reduce the impact of temporary occlusions and inaccurate detections. By assigning greater importance to highly visible viewpoints and minimizing the influence of uncertain observations, the framework improves the stability and accuracy of reconstructed motion data. Additional temporal smoothing mechanisms are also proposed to reduce high-frequency jitter and provide smoother trajectories for downstream simulation and analysis tasks.

6.2. Economic Feasibility

From a budgetary and resource allocation perspective, the MotionBridge system demonstrates excellent cost efficiency, making it highly suitable for academic institutions, student projects, and low-resource research environments. Traditional marker-based motion capture systems often require investments ranging from several thousand to tens of thousands of dollars. In contrast, the complete hardware setup for MotionBridge can be assembled for an estimated budget of approximately NPR 24,000, significantly reducing the financial barriers associated with motion capture research.

Furthermore, the entire software stack used in the project is open-source and freely available. Core technologies such as `Python`, `OpenCV`, `MediaPipe`, `PyBullet`, `Pandas`, and `Git` do not require licensing fees or recurring subscription costs. This eliminates long-term software expenses and allows future development and deployment without additional financial commitments.

6.3. Operational Feasibility

Operational feasibility assesses how effectively the proposed system can function within real-world deployment environments. The project minimizes operational complexity by clearly defining the scope of simulation and data collection components. While detailed hand and finger landmark coordinates are captured and stored within the generated datasets, only major upper-body joints are visualized in the simulation environment. This design choice helps maintain smooth execution performance while still preserving comprehensive motion data for future analysis.

The framework also generates structured datasets automatically during operation. Motion information is exported into organized CSV files containing temporal and spatial information for relevant body joints. These datasets are directly suitable for downstream applications such as imitation learning, behavioral analysis, machine learning model training, and robotics research, thereby increasing the long-term value of the system.

In addition, the architecture is highly adaptable to different environments. The system requires only standard indoor lighting conditions and an initial camera calibration procedure to establish accurate spatial references. Once calibrated, the framework can operate in ordinary indoor spaces without requiring specialized infrastructure, making deployment straightforward and practical.

6.4. Schedule Feasibility

The project timeline presents a realistic and well-structured development plan. Initial phases are dedicated to literature review, requirement analysis, and system planning, ensuring that design decisions are supported by sufficient background research. The central portion of the schedule focuses on system design, implementation, and integration activities, providing adequate time for developing and refining the core motion capture and simulation modules.

Subsequent phases allocate sufficient effort toward testing, validation, and performance evaluation under different operating conditions. Time is also reserved for documentation, report preparation, and final refinements before project submission. This balanced distribution of tasks reduces scheduling risks and supports the successful completion of project objectives within the planned timeframe.

A. APPENDICES

A.1. PROJECT BUDGET

Table A-1. Estimated Project Budget for MotionBridge

S.N.	Qty	Unit Price (NPR)	Total Price (NPR)	
1	USB Webcams	3	4000	12000
2	Camera Tripods	3	1,500	4,500
3	Powered USB Hub	1	2,000	2,000
4	Calibration Target	1	1,000	1,000
5	Software & Tools	-	-	Free
6	Project Documentation	-	2,500	2,500
7	Contingency Fund	-	2,000	2,000
			Total Estimated Budget:	**NPR 24,000**

A.2. PROJECT TIMELINE

A.2.1. Gantt Chart

Table A-2. Gantt Chart for Project Timeline

REFERENCES

[1] J. Kim, S. Park, and H. Lee, "Human pose estimation using MediaPipe pose and optimization method based on a humanoid model," *Applied Sciences*, vol. 13, no. 4, p. 2700, 2023.

[2] S. Sarkar, Y. Jang, and I. Jeong, "Multi-camera-based 3D human pose estimation for close-proximity human-robot collaboration in construction," in *Proceedings of the 9th International Conference on Construction Engineering and Project Management (ICCEPM)*, Las Vegas, NV, USA, 2022.

[3] P. Zell, B. Rosenhahn, and T. Müller, "Multimodal human motion dataset of 3D anatomical landmarks and pose keypoints," *Data in Brief*, 2024.

[4] S. Bultmann and S. Behnke, "Real-time multi-view 3D human pose estimation using semantic feedback to smart edge sensors," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[5] L. Bragagnolo, M. Terreran, D. Allegro, and S. Ghidoni, "Multi-view pose fusion for occlusion-aware 3D human pose estimation," *arXiv preprint arXiv:2408.15810*, 2024.

[6] V. Leroy, J.-S. Franco, and E. Boyer, "Probabilistic triangulation for uncalibrated multi-view 3D human pose estimation," *arXiv preprint arXiv:2309.04756*, 2023.

[7] K. Liang, Y. Chen, and Z. Wang, "Behavior imitation for manipulator control and grasping with deep reinforcement learning," *arXiv preprint arXiv:2405.01284*, 2024.

[8] X. Jiang, B. Wu, S. Li, Y. Zhu, G. Liang, Y. Yuan, Q. Li, and J. Zhang, "Multi-humanoid robot arm motion imitation and collaboration based on improved retargeting," *Biomimetics*, vol. 10, no. 3, p. 190, 2025.

[9] T. Faria, R. Sousa, and C. Santos, "Teleoperation system for service robots using a virtual reality headset and 3D pose estimation," *Sensors*, 2025.

[10] G. Moya-Alcover, A. Jaume-i Capó, and J. Varona, "Motion capture benchmark of real industrial tasks and traditional crafts for human movement analysis," *IEEE Access*, 2023.